Google Drive Upload Organizer - 10 Métodos de Upload

Este documento apresenta 10 diferentes métodos para fazer upload de arquivos para o Google Drive utilizando nosso script organizador, que classifica automaticamente arquivos em pastas específicas com base no tipo de arquivo.

Índice

1. Interface Web Integrada
2. cURL via Linha de Comando
3. Postman ou Similar
4. Script Python
5. Script Node.js
6. Formulário HTML Personalizado
7. Aplicativo Mobile
8. Google Apps Script Trigger
9. Integração com Serviços de Automação
10. Upload em Lote

1. Interface Web Integrada

O modo mais simples para usuários finais. Acesse a interface web embutida no script.

Como usar:

1. Acesse a URL do Web App após publicação: `https://script.google.com/macros/s/SEU_ID_DO_SCRIPT/exec`
2. Selecione o arquivo usando o botão de navegação
3. Clique em "Enviar"
4. Aguarde o processamento e receba o link do arquivo ao finalizar

Vantagens: Interface amigável, não requer conhecimentos técnicos.

2. cURL via Linha de Comando

Ideal para automação e script em ambientes Unix/Linux/Mac.

Linux/Mac:

```bash
curl -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "fileName": "documento.pdf",
    "fileData": "'$(base64 -w 0 caminho/para/documento.pdf)'"
  }' \
  https://script.google.com/macros/s/SEU_ID_DO_SCRIPT/exec
```

Windows (PowerShell):

```powershell
$filePath = "C:\caminho\para\documento.pdf"
$fileName = "documento.pdf"
$fileData = [Convert]::ToBase64String([IO.File]::ReadAllBytes($filePath))

$json = @{
    fileName = $fileName
    fileData = "data:application/pdf;base64,$fileData"
} | ConvertTo-Json

Invoke-RestMethod -Uri "https://script.google.com/macros/s/SEU_ID_DO_SCRIPT/exec" `
                  -Method Post `
                  -ContentType "application/json" `
                  -Body $json
```

Vantagens: Integração com scripts de automação, fácil uso em servidores.

3. Postman ou Similar

Para testes e integração de API com interface gráfica.

Como usar:

1. Abra o Postman
2. Configure uma requisição POST para `https://script.google.com/macros/s/SEU_ID_DO_SCRIPT/exec`
3. Configure o cabeçalho: `Content-Type: application/json`
4. No corpo da requisição, selecione "raw" e JSON, e adicione:

```json
{
  "fileName": "meu_arquivo.pdf",
  "fileData": "data:application/pdf;base64,BASE64_DO_ARQUIVO_AQUI"
}
```

5. Clique em "Send"

Vantagens: Interface visual para testar a API, facilita o debug.

4. Script Python

Para automatização ou integração com sistemas Python.

python

```python
import requests
import base64
import json
import os

def upload_file(file_path, script_url):
    file_name = os.path.basename(file_path)

    # Lê e codifica o arquivo em base64
    with open(file_path, "rb") as file:
        encoded_string = base64.b64encode(file.read()).decode('utf-8')

    # Prepara os dados para envio
    payload = {
        "fileName": file_name,
        "fileData": f"data:application/octet-stream;base64,{encoded_string}"
    }

    # Envia a requisição
    response = requests.post(
        script_url,
        headers={"Content-Type": "application/json"},
        data=json.dumps(payload)
    )

    return response.json()

# Exemplo de uso
result = upload_file(
    "caminho/para/arquivo.pdf",
    "https://script.google.com/macros/s/SEU_ID_DO_SCRIPT/exec"
)
print(result)
```

Vantagens: Integração com sistemas Python, possibilidade de processamento avançado.

5. Script Node.js

Para integração com sistemas JavaScript/Node.js.

```javascript
const fs = require('fs');
const axios = require('axios');

async function uploadFile(filePath, scriptUrl) {
  try {
    // Lê o arquivo e converte para base64
    const fileData = fs.readFileSync(filePath);
    const fileName = filePath.split('/').pop();
    const base64Data = fileData.toString('base64');

    // Prepara a payload
    const payload = {
      fileName: fileName,
      fileData: `data:application/octet-stream;base64,${base64Data}`
    };

    // Envia a requisição
    const response = await axios.post(scriptUrl, payload, {
      headers: { 'Content-Type': 'application/json' }
    });

    return response.data;
  } catch (error) {
    console.error('Erro ao fazer upload:', error.message);
    return { success: false, message: error.message };
  }
}

// Exemplo de uso
uploadFile(
  './caminho/para/arquivo.pdf',
  'https://script.google.com/macros/s/SEU_ID_DO_SCRIPT/exec'
).then(result => console.log(result));
```

Vantagens: Integração com sistemas Node.js, bom para ambientes JavaScript.

6. Formulário HTML Personalizado

Para integração em sites e aplicações web existentes.

html

```html
<!DOCTYPE html>
<html>
<head>
  <title>Upload para Google Drive</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 20px; }
    .container { max-width: 600px; margin: 0 auto; }
    .progress { margin-top: 20px; display: none; }
    .result { margin-top: 20px; padding: 10px; border: 1px solid #eee; }
  </style>
</head>
<body>
  <div class="container">
    <h2>Upload para Google Drive</h2>
    <form id="uploadForm">
      <input type="file" id="fileInput" required>
      <button type="submit">Enviar</button>
    </form>
    <div class="progress" id="progress">Enviando arquivo...</div>
    <div class="result" id="result"></div>
  </div>

  <script>
    document.getElementById('uploadForm').addEventListener('submit', async function(e)
      e.preventDefault();
      const fileInput = document.getElementById('fileInput');
      const progressDiv = document.getElementById('progress');
      const resultDiv = document.getElementById('result');

      if (!fileInput.files[0]) {
        alert('Selecione um arquivo primeiro.');
        return;
      }

      progressDiv.style.display = 'block';
      resultDiv.innerText = '';

      const file = fileInput.files[0];
      const reader = new FileReader();

      reader.onload = async function(e) {
        try {
          const response = await fetch(
            'https://script.google.com/macros/s/SEU_ID_DO_SCRIPT/exec',
            {
```

```
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
              fileName: file.name,
              fileData: e.target.result
            })
          }
        );

        const result = await response.json();
        progressDiv.style.display = 'none';

        if (result.success) {
          resultDiv.innerHTML = `Upload realizado com sucesso! <a href="${result.fil
        } else {
          resultDiv.innerText = `Erro: ${result.message}`;
        }
      } catch (error) {
        progressDiv.style.display = 'none';
        resultDiv.innerText = `Erro: ${error.message}`;
      }
    };

    reader.readAsDataURL(file);
  });
</script>
</body>
</html>
```

Vantagens: Personalização completa da interface, integração com sites existentes.

7. Aplicativo Mobile

Para uploads via dispositivos móveis usando um app simples.

Exemplo com React Native:

jsx

```
import React, { useState } from 'react';
import { View, Button, Text, StyleSheet } from 'react-native';
import * as DocumentPicker from 'expo-document-picker';
import * as FileSystem from 'expo-file-system';

export default function App() {
  const [status, setStatus] = useState('');

  const pickAndUploadDocument = async () => {
    try {
      // Seleciona o arquivo
      const result = await DocumentPicker.getDocumentAsync({});
      if (result.type === 'cancel') return;

      setStatus('Preparando upload...');

      // Lê o arquivo como base64
      const base64 = await FileSystem.readAsStringAsync(result.uri, {
        encoding: FileSystem.EncodingType.Base64
      });

      setStatus('Enviando arquivo...');

      // Envia para o Google Drive
      const response = await fetch('https://script.google.com/macros/s/SEU_ID_DO_SCRIP
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          fileName: result.name,
          fileData: `data:${result.mimeType};base64,${base64}`
        })
      });

      const responseData = await response.json();

      if (responseData.success) {
        setStatus(`Upload realizado com sucesso! URL: ${responseData.fileUrl}`);
      } else {
        setStatus(`Erro: ${responseData.message}`);
      }
    } catch (error) {
      setStatus(`Erro: ${error.message}`);
    }
  };
```

```
  return (
    <View style={styles.container}>
      <Button title="Selecionar e Enviar Arquivo" onPress={pickAndUploadDocument} />
      <Text style={styles.status}>{status}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 20,
  },
  status: {
    marginTop: 20,
    textAlign: 'center',
  },
});
```

Vantagens: Acesso via dispositivos móveis, experiência de usuário dedicada.

8. Google Apps Script Trigger

Para uploads automáticos dentro do ecossistema Google.

javascript

```
/**
 * Script para copiar arquivos de uma pasta de origem para o nosso organizador
 */
function monitorarEOrganizarArquivos() {
  // ID da pasta que você quer monitorar
  const PASTA_ORIGEM_ID = 'ID_DA_SUA_PASTA_ORIGEM';

  // URL do Web App do organizador
  const ORGANIZADOR_URL = 'https://script.google.com/macros/s/SEU_ID_DO_SCRIPT/exec';

  // Obtém a pasta de origem
  const pastaOrigem = DriveApp.getFolderById(PASTA_ORIGEM_ID);

  // Obtém todos os arquivos na pasta
  const arquivos = pastaOrigem.getFiles();

  while (arquivos.hasNext()) {
    const arquivo = arquivos.next();

    // Lê o conteúdo do arquivo
    const blob = arquivo.getBlob();
    const conteudo = Utilities.base64Encode(blob.getBytes());

    // Envia para o organizador
    const payload = {
      fileName: arquivo.getName(),
      fileData: `data:${blob.getContentType()};base64,${conteudo}`
    };

    const opcoes = {
      method: 'post',
      contentType: 'application/json',
      payload: JSON.stringify(payload)
    };

    try {
      const response = UrlFetchApp.fetch(ORGANIZADOR_URL, opcoes);
      const result = JSON.parse(response.getContentText());

      if (result.success) {
        // Opcional: remover o arquivo original após organização
        // arquivo.setTrashed(true);
        Logger.log(`Arquivo ${arquivo.getName()} organizado com sucesso`);
      }
    } catch (erro) {
```

```
      Logger.log(`Erro ao organizar ${arquivo.getName()}: ${erro.message}`);
    }
  }
}

// Configure um gatilho para executar periodicamente
function configurarGatilho() {
  ScriptApp.newTrigger('monitorarEOrganizarArquivos')
    .timeBased()
    .everyHours(1)  // Execute a cada hora
    .create();
}
```

Vantagens: Integração total com o ecossistema Google, processamento automático.

9. Integração com Serviços de Automação

Para integração com plataformas como Zapier, IFTTT, etc.

Configuração no Zapier:

1. **Trigger (gatilho)**: Escolha um gatilho (ex: Novo arquivo no Dropbox)
2. **Action (ação)**: Selecione "Webhooks by Zapier" > "POST"
3. **Configuração da ação:**
   - URL: `https://script.google.com/macros/s/SEU_ID_DO_SCRIPT/exec`
   - Payload Type: JSON
   - Data:

     ```
     {
       "fileName": "{{file_name}}",
       "fileData": "{{file_base64}}"
     }
     ```

   - Headers: `{"Content-Type": "application/json"}`

Vantagens: Conecta com centenas de serviços sem código.

10. Upload em Lote

Para enviar múltiplos arquivos de uma vez.

bash

```bash
#!/bin/bash
# Script para upload em lote de arquivos para o Google Drive

SCRIPT_URL="https://script.google.com/macros/s/SEU_ID_DO_SCRIPT/exec"
DIRETORIO_ARQUIVOS="./arquivos_para_upload/"

# Verifica se o diretório existe
if [ ! -d "$DIRETORIO_ARQUIVOS" ]; then
  echo "Diretório $DIRETORIO_ARQUIVOS não encontrado!"
  exit 1
fi

# Processa cada arquivo no diretório
for arquivo in "$DIRETORIO_ARQUIVOS"*; do
  if [ -f "$arquivo" ]; then
    nome_arquivo=$(basename "$arquivo")
    echo "Enviando $nome_arquivo..."

    # Codifica o arquivo em base64
    conteudo_base64=$(base64 -w 0 "$arquivo")

    # Determina o tipo MIME aproximado
    mime_type=$(file --mime-type -b "$arquivo")

    # Prepara e envia a requisição
    curl -s -X POST \
      -H "Content-Type: application/json" \
      -d '{
        "fileName": "'"$nome_arquivo"'",
        "fileData": "data:'"$mime_type"';base64,'"$conteudo_base64"'"
      }' \
      "$SCRIPT_URL" > /dev/null

    echo "✓ $nome_arquivo enviado"
    # Aguarda um pouco para não sobrecarregar
    sleep 1
  fi
done

echo "Processo concluído!"
```

Vantagens: Processa múltiplos arquivos em sequência, ideal para migrações.

Considerações Importantes

- **Limites do Google Apps Script**:
  - Tempo máximo de execução: 6 minutos
  - Tamanho máximo de payload: ~50MB
  - Cota diária de execuções
- **Segurança**:
  - Configure adequadamente as permissões do Web App
  - Considere adicionar autenticação para uploads sensíveis
  - Evite armazenar informações sensíveis no script
- **Performance**:
  - Arquivos muito grandes podem causar timeouts
  - Em caso de muitos uploads, considere limitar a taxa de envio

---

Personalização e Suporte

Para personalizar este script para suas necessidades específicas ou obter suporte, entre em contato pelo email: [seu-email@exemplo.com]